



Python应用基础

第7章 用文件存取数据

陈刚

中山大学岭南学院

基础篇目录

Contents

第 1 章

概述

第 2 章

Python 基本操作

第 3 章

列表和元组

第 4 章

控制结构

第 5 章

字典和集合

第 6 章

用函数打包代码

第 7 章

用文件存取数据

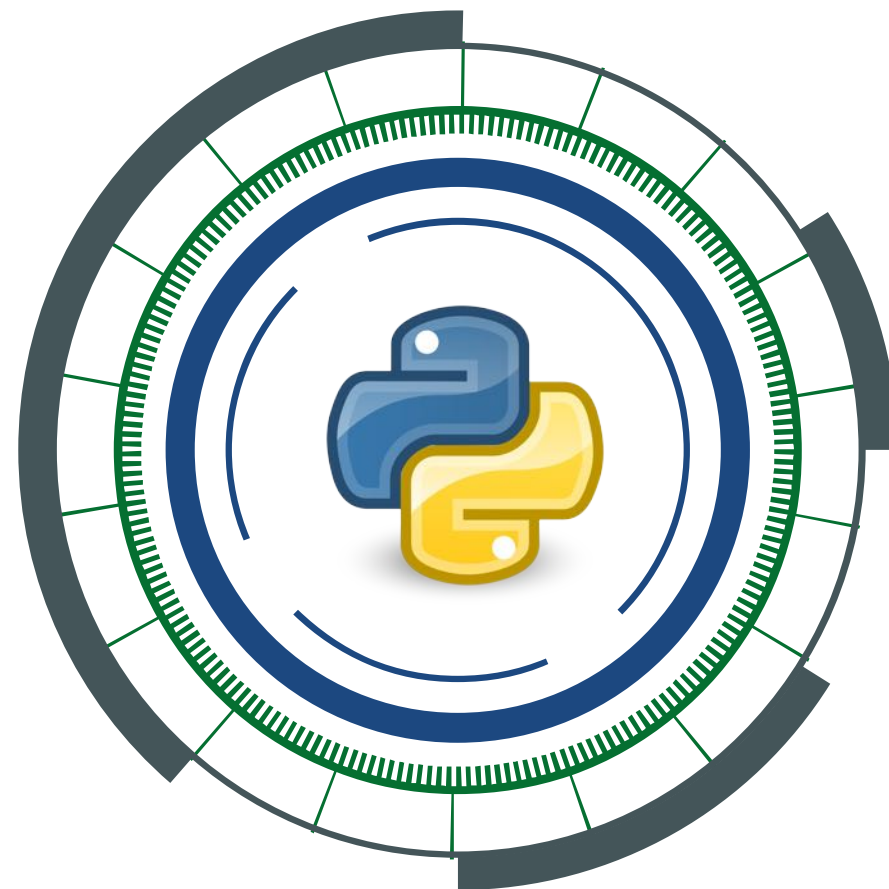


pythonTM
programming



学习目标

- 1 掌握文件与路径的概念
- 2 熟悉文本文件与二进制文件的区别
- 3 熟练掌握文件打开、关闭、读取和写入操作
- 4 掌握 with 语句的用法
- 5 掌握 pickle 文件的用法
- 6 了解 JSON 格式的用法
- 7 灵活运用文件解决实际问题





文件与路径

文件

- **文件**指的是存储在磁盘上的数据序列，它可以包含任何数据内容。
- **文件名**通常由文件主名和扩展名构成。
主名和扩展名之间用一个圆点“.”隔开。
扩展名由 1 ~ 4 个字符组成，用于表示文件的类型。
- 计算机中使用“**文件夹**”（也常常称为目录）来组织文件。文件夹可以包含文件和其他文件夹。

路径

- **路径**指明文件在计算机中存储的位置。路径中包含了存储文件的各级文件夹。文件夹之间用斜线隔开。
- 路径分隔符：
Windows 中使用倒斜杠“/”来分隔
在Mac OS X 和 Linux 中使用正斜杠“\”
- 有两种方式表示文件路径：
 - 绝对路径：从根目录开始的路径
 - 相对路径：从当前工作目录开始的

在 macOS 的 Terminal (bash) 中，使用以下命令查看当前路径：

```
bash
```

```
pwd
```

(pwd 是 "print working directory" 的缩写)

示例：

```
bash
```

```
$ pwd
```

```
/Users/yourusername/Documents
```

系统/Shell	命令
macOS/Linux (bash)	pwd
Windows (cmd)	cd
Windows (PowerShell)	pwd 或 Get-Location

路径

- 每个运行在计算机上的程序，都有一个“当前工作目录”。
在 Python 中，可以用 `os` 模块中的 `getcwd()` 方法获取当前工作目录：

```
❶ import os  
❷ os.getcwd()  
  
'/Users/'
```

- 假如在根目录的 `Users` 文件夹下有一个文件夹 `Documents` 文件夹，存储了 `report.docx` 文件。
 - 绝对路径描述： `/Users/Documents/report.docx`
 - 相对路径描述： `Documents/report.docx`

文件相关操作

- `listdir(path=None)` 方法返回指定目录中包含的文件和子目录列表，默认情况下返回当前工作目录中的信息。

```
❶ os.listdir('data/sales')  
['2020_04.csv', '2020_01.csv', '2020_02.csv', '2020_03.csv']
```

- `exists(path)` 用于检验文件是否存在，其中 `path` 是文件或文件夹的路径。

```
❷ print(os.path.exists('data/sales/2020_01.csv'))  
❸ print(os.path.exists('data/sales/2020_12.csv'))  
  
True  
False
```


课堂小练习：「我是谁？我在哪？」

运行以下代码，并回答"我的 Python 程序现在在哪个文件夹里？"

```
import os  
print("我现在在：", os.getcwd())
```

加分挑战：用 `os.listdir()` 看看这个文件夹里有什么"邻居"文件

• 当前工作目录相关

`os.getcwd()` # 获取当前工作目录
`os.chdir('/path/to/dir')` # 改变当前工作目录

• 目录内容列出

`os.listdir()` # 列出当前目录
`os.listdir('/specific/path')` # 列出指定目录

• 路径存在性检查

`os.path.exists('/path')` # 路径是否存在
`os.path.isfile('/path')` # 是否是文件
`os.path.isdir('/path')` # 是否是目录
`os.path.islink('/path')` # 是否是符号链接

• 路径操作

`os.path.abspath('file.txt')` # 获取绝对路径
`os.path.realpath('link')` # 解析符号链接的真实路径
`os.path.dirname('/a/b/c.txt')` # 返回 '/a/b'
`os.path.basename('/a/b/c.txt')` # 返回 'c.txt'
`os.path.join('dir', 'file')` # 跨平台拼接路径
`os.path.split('/a/b/c.txt')` # 返回 ('/a/b', 'c.txt')
`os.path.splitext('file.txt')` # 返回 ('file', '.txt')

• 目录操作

`os.mkdir('newdir')` # 创建单级目录
`os.makedirs('a/b/c')` # 创建多级目录
`os.rmdir('emptydir')` # 删除空目录
`os.removedirs('a/b/c')` # 递归删除空目录

• 文件操作

`os.remove('file.txt')` # 删除文件
`os.rename('old.txt', 'new.txt')` # 重命名/移动文件
`os.replace('src.txt', 'dst.txt')` # 替换文件 (跨平台)

• 文件和目录信息

`os.stat('file.txt')` # 获取文件状态 (大小、修改时间等)
`os.path.getsize('file.txt')` # 获取文件大小 (字节)
`os.path.getmtime('file.txt')` # 获取修改时间戳
`os.path.getctime('file.txt')` # 获取创建时间

• 权限和属性

`os.chmod('file.txt', 0o755)` # 修改权限
`os.chown('file.txt', uid, gid)` # 修改所有者 (Unix/macOS)
`os.access('file.txt', os.R_OK)` # 检查读权限



文本文件与二进制文件

文本文件与二进制文件

- 计算机上存储的文件都是以二进制存储的，一般可以采取以下文本文件和二进制文件方式读取。
- **文本文件**：文件内容是常规的字符，不会包含字体、大小和颜色等信息，例如带有 .py 扩展名的 Python 源文件。
- **二进制文件**：文件内容是特殊的字节数据，例如 Word 文档、PDF 文档、图像文件和编译后的可执行文件。

字符集

- 最常用的美国标准信息交换代码（American Standard Code for Information Interchange, **ASCII**码）定义了从 0 到 127 的整数对应的字符。

```
❶ print(ord('a'))  
❷ print(chr(97))  
97  
a
```

ord() — 获取字符的 Unicode 码点

全称：ordinal（序数）

功能：输入一个单字符字符串，返回它对应的 Unicode 整数编号。

记忆：ord → 想成 "order"（顺序号）

chr() — 从 Unicode 码点获取字符

全称：character（字符）

功能：输入一个整数（有效 Unicode 码点），返回它对应的字符。

拼写：chr → 就是 "char"（字符）缺了 a

- **Unicode** 突破了ASCII码只能表示英文字母和符号的限制，为每种语言的每个字符设定了唯一的编码，以满足跨语言、跨平台进行文本转换和处理的要求。

```
❸ ord('你')  
20320
```

字符集

- 将 Unicode 字符按照特定编码规则（例如 UTF8 编码）翻译为原始字节形式的过程被称为**编码**。反过来，把原始字节翻译为 Unicode 字符串的过程则被称为**解码**。
- 文本文件和二进制文件的主要区别在于文本文件有统一字符编码，但二进制文件没有。当打开一个文本文件时，读取数据会自动将其内容解码，并将解码的内容返回为一个字符串；向文件中写入数据时，会将写入的字符串自动编码。

在这一部分，我们主要讨论的是如何利用Python读写文本文件。

课堂小练习：「猜猜我是谁」

- 以下代码，请预测输出：

```
python  
print(ord('A'))  
print(chr(65))  
print(ord('帅'))
```

- 输入自己名字的第一个字，用 `ord()` 转换成数字，然后在班里"猜数字"——看谁的 Unicode 码最大！



操作文件

操作文件的步骤

● 在 Python 中，操作文件包含 3 个步骤：

- (1) 调用 `open()` 函数打开物理文件，返回一个文件对象。
- (2) 调用文件对象的 `read()` 或 `write()` 方法读写文件。
- (3) 调用文件对象关闭文件。

打开文件

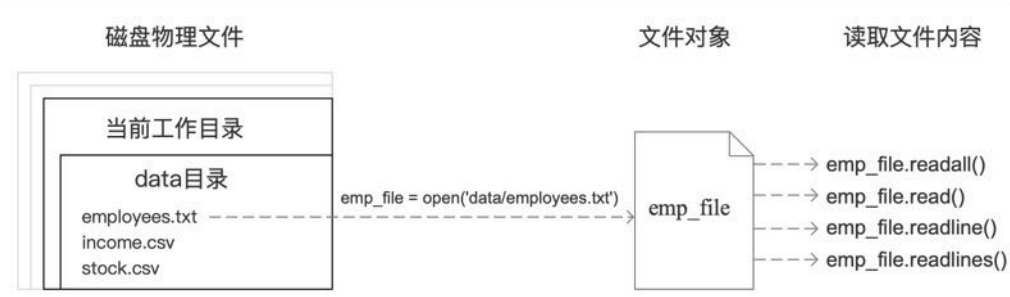
- 在读写文件内容前，需要创建文件对象指向磁盘中的物理文件。

Python 通过内置的 `open()` 函数打开文件并创建该对象。

```
open(file, mode='r', encoding=None)
```

参数 `file` 是需要打开的物理文件的绝对路径或相对路径字符串。

```
❶ emp_file = open('data/employees.txt')
❷ type(emp_file)
<class '_io.TextIOWrapper'>
❸ emp_file
<_io.TextIOWrapper name='data/employees.txt' mode='r'
encoding='UTF-8'>
```



`_io.TextIOWrapper` 是 Python 中用于文本文件读写的对象类型。

打开文件

● 参数 mode 是打开文件的模式，如表所示，默认值为 'r'。

文件打开模式

模式	说明
'r'	默认值，读模式，文件不存在触发 FileNotFoundError 异常
'w'	写模式，文件不存在则创建，存在则覆盖
'x'	写模式，文件不存在则创建，存在则触发 FileExistError 异常
'a'	追加模式，文件不存在则创建，存在则在文件末尾追加内容
'b'	以二进制方式打开文件
't'	默认值，以文本文件方式打开文件
'+'	结合 r/w/x/a 使用，创建可以同时读写的文件对象

```
# 示例1: 读取文件
emp_file = open('data/employees.txt', 'r', encoding='utf-8')
content = emp_file.read()
print(content)
emp_file.close() # 重要: 用完要关闭
```

```
# 示例2: 写入文件
emp_file = open('data/employees.txt', 'w', encoding='utf-8')
emp_file.write('张三, 30, 工程师\n')
emp_file.write('李四, 25, 设计师')
emp_file.close()
```

```
# 示例3: 追加内容
emp_file = open('data/employees.txt', 'a', encoding='utf-8')
emp_file.write('\n王五, 35, 经理')
emp_file.close()
```

打开文件

- 参数encoding是指定文本文件读取数据时解码或写入数据时编码所用的规则，为默认值 None 时使用的规则取决于操作系统。通过 sys 模块的方法可以查看当前默认编码。

```
❶ import sys
❷ sys.getdefaultencoding()
'utf-8'
```

通常来说，在Mac OS X和Linux系统中默认编码为utf-8。在中文Windows系统中默认编码为gbk。

打开文件时的编码格式要与文本本身使用编码格式一致。

```
❸ emp_file = open('data/employees.txt', mode='a', encoding='utf-8')
❹ emp_file
<_io.TextIOWrapper name='data/employees.txt' mode='a' encoding='utf-8'>
```

课堂小练习：「文件保险箱」

- 创建一个秘密日记文件：

```
python
```

```
secret = open('my_secret.txt', 'w', encoding='utf-8')
```

```
secret.write("我的秘密：我今天上课学会了一招! 🐱 ")
```

```
secret.close()
```

- 问答：如果再用 'w' 模式运行一次，会发生什么？那用 'a' 呢？

关闭文件

- 在操作完文件后，必须将文件关闭，这点非常重要。
- 调用文件对象的 `close()` 方法可以关闭文件，终止文件对象与磁盘物理文件的连接、释放操作系统资源。

```
❶ emp_file.close()
```

写入文本文件

- 当指定 `open()` 函数的 `mode` 参数值为 '`w`'、'`x`' 和 '`a`' 时，可以向创建的文件对象中写入数据。
- 使用文件对象的 `write()` 方法就可以将文本写入文件中。

```
emp_file.write(text)
```

其中，`text` 是要写入文件的字符串。该方法会返回写入文件的字符数量。

写入文本文件

【例】 用户输入商品名称和销售额，将输入的明细和合计信息存储到文件 sales.txt 中

```
① sales = []
② sale_amt = 0
③ while True:
    sale_info = input('请输入商品名称和销售额（用逗号隔开，直接回车结束）：')
    ④ if len(sale_info) == 0:
    ⑤     break
    goods_name, amount = sale_info.split(',')
    ⑥ sale_amt = sale_amt + float(amount)
    sales.append([goods_name, amount])
⑦ sale_file = open('sales.txt', 'w', encoding='utf-8')
⑧ sale_file.write('销售明细 :\n')
⑨ for goods_name, amount in sales:
    ⑩ sale_file.write(goods_name + ',' + amount + '\n')
sale_file.write('销售额合计: {:.2f} 元 \n'.format(sale_amt))
sale_file.close()
```



请输入商品名称和销售额（用逗号隔开，直接回车结束）：笔记本,1000

请输入商品名称和销售额（用逗号隔开，直接回车结束）：钢笔,800

请输入商品名称和销售额（用逗号隔开，直接回车结束）：铅笔,500

请输入商品名称和销售额（用逗号隔开，直接回车结束）：

课堂小练习：「班级留言板」

- 每个同学把自己最喜欢的 Python 一句话写入同一个文件：

```
with open('class_quotes.txt', 'a', encoding='utf-8') as f:
```

```
    name = input("你的大名：")
```

```
    quote = input("你最喜欢的一句 Python 心得：")
```

```
    f.write(f"{name} 说： {quote}\n")
```

读取文本文件

- 使用 `open()` 函数打开文件时，`mode` 参数的默认值是“rt”，即以文本文件的形式读取文件。
- 对于可读的文件对象，可以使用以下四种内置方法读取内容：

文件读取方法	
方法	说明
<code>readall()</code>	以字符串形式返回整个文件内容
<code>read(size=-1)</code>	默认读入从当前位置至文件末尾的内容；当 <code>size</code> 参数为大于 0 的正整数 <code>n</code> 时，从文件中读入最多 <code>n</code> 个字符
<code>readline(size=-1)</code>	默认从文件中读入一行内容；当 <code>size</code> 参数为大于 0 的正整数 <code>n</code> 时，从当前行读入最多 <code>n</code> 个字符
<code>readlines(hint=-1)</code>	默认返回以文件中所有行为元素构成的列表；当 <code>hint</code> 参数为大于 0 的正整数 <code>n</code> 时，读入的所有行字符数不超过 <code>n</code>

读取文本文件

- 当打开文本文件时，会有一个指针指向文件开头的字符。每次读取数据后，指针会往后移动到相应位置。直到指向文件尾部标记 EOF。

```
❶ in_file = open('sales.txt',  
❷ encoding='utf-8')  
   print(in_file.readline())
```

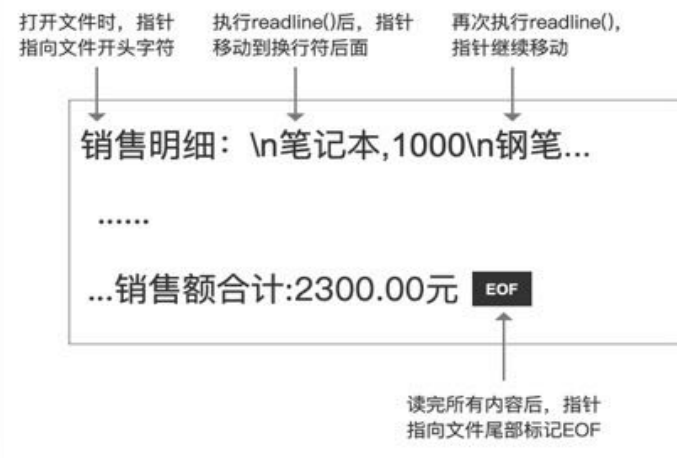
销售明细:

```
❸ print(in_file.readline())
```

笔记本,1000

```
❹ in_file.seek(0)  
   in_file.readline()
```

'销售明细:\n'



文件内容全部读取后，指针指向文件尾部。如果需要从头读取文件内容，需要通过文件对象的 `seek()` 方法将指针移动到文件开头，或者使用 `open()` 函数重新打开文件。

读取文本文件

- 使用 for 循环逐行读取。在绝大多数情况下，对于文本文件的读取都是逐行读取。文件对象本身是一个可迭代对象。因此，推荐使用 for 循环来逐行读取文本文件内容。

```
❶ in_file = open('sales.txt',  
❷ encoding='utf-8')  
❸ for line in in_file:  
❹ print(line, end='')  
in_file.close()
```

```
销售明细：  
笔记本,1000  
钢笔,800  
铅笔,500  
销售额合计：2300.00 元
```

特别是在读取非常大的文本文件时，使用这种方式不会一次性将整个文件读入内存中，而是逐行读取，从而减轻内存压力。

读取文本文件

- 文件作为可迭代对象， 可以使用列表推导式和 `map()` 函数对每行进行相应处理， 生成新的列表。

```
❶ in_file = open('sales.txt', encoding='utf-8')  
❷ lines = [line.rstrip() for line in in_file]  
❸ print(lines)  
❹ in_file.close()
```

['销售明细:', '笔记本,1000', '钢笔,800', '铅笔,500', '销售额合计:
2300.00 元']

```
❺ in_file = open('sales.txt', encoding='utf-8')  
❻ lines = [line.rstrip() for line in in_file]
```

['销售明细:', '笔记本,1000', '钢笔,800', '铅笔,500', '销售额合计:
2300.00 元']

课堂小练习：「猜谜翻牌」

- 提前准备一个谜语文件 riddle.txt:

谜面：什么东西越洗越脏？

答案：水

```
with open('riddle.txt', 'r', encoding='utf-8') as f:
```

```
    print(f.readline()) # 只显示谜面
```

```
    input("按回车查看答案...")
```

```
    print(f.readline()) # 显示答案
```

使用 with 语句自动管理

- **with** 语句适用于对资源进行访问的场合，确保不管使用过程中是否发生异常都会释放资源，比如文件使用后自动关闭。

```
❶ with open('sales.txt', encoding='utf-8') as  
❷ sales_file:  
❸ for line in sales_file:  
    print(line, end='')
```

销售明细：

笔记本,1000

钢笔,800

铅笔,500

销售额合计：2300.00 元

课堂小练习：「懒人福音」

- 对比演示：

```
python
```

```
# 传统写法
```

```
f = open('test.txt', 'w')
```

```
f.write("hello")
```

```
# 忘了写 close() 会怎样?
```

```
# 优雅写法
```

```
with open('test.txt', 'w') as f:
```

```
    f.write("hello")
```

```
# 自动关闭，不会忘!
```


课堂综合大挑战：「小明的账单助手」

背景：小明的生活费管理程序需要保存数据。

任务：编写程序实现以下功能：

- 用户可以输入：商品名称和金额
- 每次输入后自动保存到文件
- 可以查看历史账单
- 可以统计总花费

加分彩蛋：

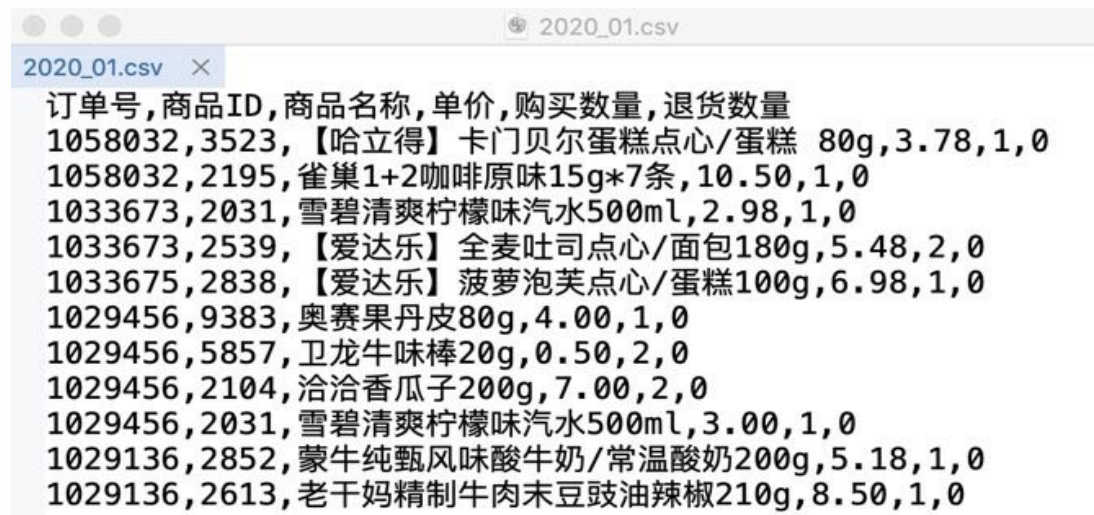
- 添加删除账单功能
- 添加按日期查询
- 把文件保存到桌面上



案例：销售统计

案例：销售统计

【例】某便利店每个月的销售数据，按照年份和月份命名的方式存储在data/sales目录下。例如2020_01.csv是2020年1月的销售数据。文件的第一行是标题，从第二行开始是具体销售数据，数据之间用逗号隔开，如图所示。



```
2020_01.csv
订单号,商品ID,商品名称,单价,购买数量,退货数量
1058032,3523,【哈立得】卡门贝尔蛋糕点心/蛋糕 80g,3.78,1,0
1058032,2195,雀巢1+2咖啡原味15g*7条,10.50,1,0
1033673,2031,雪碧清爽柠檬味汽水500ml,2.98,1,0
1033673,2539,【爱达乐】全麦吐司点心/面包180g,5.48,2,0
1033675,2838,【爱达乐】菠萝泡芙点心/蛋糕100g,6.98,1,0
1029456,9383,奥赛果丹皮80g,4.00,1,0
1029456,5857,卫龙牛味棒20g,0.50,2,0
1029456,2104,洽洽香瓜子200g,7.00,2,0
1029456,2031,雪碧清爽柠檬味汽水500ml,3.00,1,0
1029136,2852,蒙牛纯甄风味酸牛奶/常温酸奶200g,5.18,1,0
1029136,2613,老干妈精制牛肉末豆豉油辣椒210g,8.50,1,0
```

现需要统计每个月的销售额以及总销售额，并将统计结果写入文本文件 statistic.txt 中。

以纯文本形式按行存储表格数据，每行的值之间使用逗号分隔，是一种通用的文件格式，称为 csv (comma-separated values) 文件。Python 提供了丰富的第三方库对这类文件进行操作，例如 csv 模块和 pandas 等。

案例：销售统计

```
import os
① stat_dic = {}
file_list = [name for name in os.listdir('data/sales') if name.endswith('.csv')]
② for file_name in file_list:
③     with open(os.path.join('data/sales', file_name), encoding='utf-8') as sales_file:
        first_line_flag = True
        month_amount = 0
        for line in sales_file:
④             if first_line_flag:
                    first_line_flag = False
                    continue
⑤                 rec = line.rstrip().split(',')
⑥                 month_amount += float(rec[-3]) * (int(rec[-2]) - int(rec[-1]))
        month_name = file_name[:4] + '年' + file_name[5:7] + '月'
⑦         stat_dic[month_name] = month_amount

⑧ stat_list = list(stat_dic.items())
stat_list.sort(key=lambda x: x[0])
out_file = open('statistic.txt', 'wt', encoding='utf-8')
total_amount = 0
⑨ for month, amount in stat_list:
        total_amount += amount
⑩     out_file.write('{} : {:.2f} 元 \n'.format(month, amount))
out_file.write('销售总额 : {:.2f} 元 \n'.format(total_amount))
out_file.close()
```

2020年01月 : 612,361.06
元
2020年02月 : 16,194.54
元
2020年03月 : 707,344.34
元
2020年04月 : 727,424.93
元
销售总额 : 2,063,324.87
元



使用 PICKLE 存储 PYTHON对象

使用 pickle 存储 Python 对象

- 在 Python 中，提供的 `pickle` 模块能够将 Python 对象直接存储到文件中。在需要使用数据时，直接从文件中读取，并还原为 Python 对象。
- `pickle` 操作的不是文本文件，而是二进制文件。
- 将 Python 对象存储到 pickle 文件的语法是：`pickle.dump(obj, file)`
- 从 pickle 文件中将二进制数据读取出来，重建为 Python 对象的语法是：

`pickle.load(file)`

使用 pickle 存储 Python 对象

【例】上一节案例中，将统计信息的 stat_dic 字典使用 pickle 模块存储在二进制文件 statistic.pkl 中，然后再次从文件中读取数据，重建为字典后，打印每个月的销售额。

```
❶ import os,pickle
stat_dic = {}
file_list = [name for name in os.listdir('data/sales') if name.endswith('.csv')]
for file_name in file_list:
    with open(os.path.join('data/sales', file_name), encoding='utf-8') as sales_file:
        first_line_flag = True
        month_amount = 0
        for line in sales_file:
            if first_line_flag:
                first_line_flag = False
                continue
            rec = line.rstrip().split(',')
            month_amount += float(rec[-3]) * (int(rec[-2]) - int(rec[-1]))
            month_name = file_name[:4] + '年' + file_name[5:7] + '月'
            stat_dic[month_name] = month_amount
❷ out_file = open('statistic.pkl', 'wb')
❸ pickle.dump(stat_dic, out_file)
❹ out_file.close()
❺ in_file = open('statistic.pkl', 'rb')
❻ read_dic = pickle.load(in_file)
❼ for month, amount in read_dic.items():
❽     print('{}销售额: {:.2f}元 '.format(month, amount))
```

2020年04月销售额: 663,464.03
元
2020年01月销售额: 566,218.23
元
2020年02月销售额: 12,538.74
元
2020年03月销售额: 650,070.04
元



使用 JSON 格式存储 PYTHON 对象

使用 JSON 格式存储 Python 对象

- **JSON** (javascript object notation) 是一种和语言无关的轻量级数据交换格式，采用文本格式来存储和表示数据。这种格式便于阅读和编写，也易于程序的解析和生成。
 - JSON 独立于编程语言，常用于网络数据交换。
 - JSON 语法规则与 Python 中的字典和列表非常相似：
 - (1) 利用一对方括号 [] 表示数组；
 - (2) 利用一对花括号 {} 表示对象；
 - (3) 利用冒号分割键值对；
 - (4) 利用逗号分隔数组的元素或对象的键值对。
- 在 JSON 格式的文本中，只能使用双引号表示字符值。

使用 JSON 格式存储 Python 对象

- 通过第三方库 `json` 实现 JSON 格式字符串与 Python 字典、列表相互转换。

- 将 Python 对象转换为 JSON 格式字符串的语法是：

```
json.dumps(obj, ensure_ascii=True)
```

`obj` 是需要转换为 JSON 格式字符串的 Python 对象，`ensure_ascii` 默认值为 `True`。如果对象中含有中文字符，应将该参数设置为 `False`，否则返回字符串中的中文为 Unicode 字节编码，该方法返回结果则是转换后的字符串。

- 将 JSON 格式字符串转换为 Python 对象的语法是：

```
json.loads(s)
```

`s` 是 JSON 格式字符串，该方法返回结果是对应的 Python 对象。

使用 JSON 格式存储 Python 对象

【例】将上一节例子中的字典 `stat_dic` 转换为 JSON 格式字符串的代码如下所示：

```
❶ import json
❷ stat_dic = {'2020年04月': 663464.03, '2020年01月': 566218.23,
              '2020年02月': 12538.74, '2020年03月': 650070.04}
❸ json_str = json.dumps(stat_dic, ensure_ascii=False)
❹ print(type(json_str))
❺ print(json_str)
<class 'str'>
{"2020年04月": 663464.03, "2020年01月": 566218.23, "2020年02月": 12538.74, "2020年03月": 650070.04}
```

相反，将上面代码中的 `json_str` 转换为 Python 字典的代码如下所示：

```
❶ converted_dic = json.loads(json_str)
❷ print(type(converted_dic))
❸ print(converted_dic)
<class 'dict'>
{'2020年04月': 663464.03, '2020年01月': 566218.23, '2020年02月': 12538.74, '2020年03月': 650070.04}
```

知识点速记表

函数/方法	作用	趣味口诀
open()	打开文件	"芝麻开门"
write()	写入	"写作业"
read()	读取	"读课文"
close()	关闭	"关门大吉"
with	自动管理	"带个自动保姆"
'w' 模式	覆盖写	"重写历史"
'a' 模式	追加写	"续写故事"
'r' 模式	只读	"只许看不许摸"

Happy coding! 🐍



第1题：学生成绩管理系统

编写学生成绩管理程序，实现以下功能：

- 从文件 scores.txt 读取学生姓名和成绩（格式：姓名,成绩）
- 计算全班平均分、最高分、最低分
- 统计各分数段的人数（0-59, 60-69, 70-79, 80-89, 90-100）
- 将统计结果写入 report.txt 文件

示例输入文件 scores.txt

张三,85

李四,92

王五,67

赵六,78

小明,45

小红,98

示例输出文件 report.txt

===== 成绩统计报告 =====

平均分：77.50

最高分：98 (小红)

最低分：45 (小明)

分数段分布：

0-59分: 1人

60-69分: 1人

70-79分: 2人

80-89分: 1人

90-100分: 1人

第2题：通讯录管理系统

开发简单的通讯录程序，支持以下操作：

- 添加联系人：输入姓名、电话、邮箱，保存到文件
- 查找联系人：输入姓名，显示完整信息
- 删除联系人：输入姓名，从文件中删除
- 显示所有联系人：按姓名排序显示
- 退出：保存并退出

数据格式

文件 contacts.txt 每行存储一个联系人，格式：
姓名,电话,邮箱

示例交互

===== 通讯录管理系统 =====

1. 添加联系人
 2. 查找联系人
 3. 删除联系人
 4. 显示所有联系人
 5. 退出
- 请选择操作(1-5): 1

姓名: 张三

电话: 13800138000

邮箱: zhangsan@example.com

 联系人已添加!

1. 添加联系人
 2. 查找联系人
 3. 删除联系人
 4. 显示所有联系人
 5. 退出
- 请选择操作(1-5): 4

===== 通讯录列表 =====

姓名: 张三 | 电话: 13800138000 | 邮箱: zhangsan@example.com

第3题：单词学习助手

设计一个英语单词学习程序，帮助用户背单词
功能要求

- 导入单词库：从 words.txt 读取单词和释义（格式：apple,苹果）
- 随机测验：随机显示英文单词，让用户输入中文释义
- 记录错题：答错的单词加入"错题本"，存入 mistakes.txt
- 复习错题：专门练习错题本中的单词
- 统计报告：显示本次学习正确率

单词库文件 words.txt

apple,苹果

book,书

computer,电脑

run,跑

beautiful,美丽的

示例运行

===== 单词小助手 =====

1. 开始测验
 2. 复习错题本
 3. 查看统计
 4. 退出
- 选择: 1

第1题: apple
你的答案: 苹果
✅ 正确!

第2题: book
你的答案: 书
✅ 正确!

第3题: computer
你的答案: 电脑
✅ 正确!

🎉 本次测验完成!

正确: 3题, 错误: 0题, 正确率: 100.0%

第4题：购物车系统

模拟电商购物车功能，功能要求如下：

- 商品管理
 - 从 products.txt 加载商品（格式：编号,名称,价格）
 - 显示所有商品列表
- 购物车操作
 - 添加商品到购物车（输入编号和数量）
 - 修改购物车中商品数量
 - 删除购物车中的商品
 - 查看购物车（显示商品名、单价、数量、小计）
- 订单结算
 - 计算总金额
 - 输入收货信息（姓名、电话、地址）
 - 生成订单文件 order_时间戳.txt

商品数据 products.txt

1,华为Mate60,6999
2,iPhone15,5999
3,小米14,3999
4,联想笔记本,5499
5,鼠标,99

示例输出（查看购物车）

===== 购物车 =====

商品名	单价	数量	小计
华为Mate60	6999	2	13998
鼠标	99	3	297

总计: 14295 元